

D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji.

(An Empowered Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Department of Computer Science & Engineering

2026-2027

AI Tools for Software Development Project Report On

InvoiceBridge: A Multi-Country Invoice Conversion and Indian GST Compliance SaaS Platform

Under The Guidance Of

Dr. P. M. Gavali

Submitted By:

1. 23UCS058 Chaitanya Bharat Khandagale

Dr. P. M. Gavali
Project Guide

Prof. (Dr.) D.V.Kodavade
Head of Department

External Examiner

INDEX

1 Introduction
- 1.1 Project Overview and Problem Statement
- 1.2 Existing Project and Limitations
- 1.3 Objectives
- 1.4 Proposed AI-Assisted Approach

2 Requirement Analysis
- 2.1 Functional and Non-Functional Requirements
- 2.2 User Stories and Acceptance Criteria
- 2.3 Prompts, AI Outputs and Refinement

3 System Design
- 3.1 System Architecture and Module Design
- 3.2 User Interface Design / Wireframes
- 3.3 Design Prompts, AI Outputs and Final Design

4 Coding and Implementation
- 4.1 Development Environment and Technology Stack
- 4.2 AI-Assisted Code Generation
- 4.3 Debugging, Refactoring and Code Optimization
- 4.4 Coding Prompts, Generated Code and Manual Modifications

5 Testing
- 5.1 Testing Strategy and Test Plan
- 5.2 AI-Generated Test Cases
- 5.3 Unit, Integration and System Testing
- 5.4 Defect Identification and AI-Assisted Debugging
- 5.5 Testing Prompts and Actual Results

6 Deployment and Documentation
- 6.1 Deployment and Execution Procedure
- 6.2 Deployment Prompts and Configuration
- 6.3 User Manual and Technical Documentation

7 References

8 Appendices
1. Appendix A: Complete Prompt Dictionary and Prompt History
2. Appendix B: AI Tool Usage Log
3. Appendix C: Important Source Code and Screenshots
4. Appendix D: Test Cases and Execution Evidence

1. Introduction

1.1 Project Overview and Problem Statement

Project Overview

InvoiceBridge: A Multi-Country Invoice Conversion and Indian GST Compliance SaaS Platform is a full-stack web application developed to simplify the conversion of foreign invoices into Indian GST-formatted tax invoices. The project focuses on reducing the manual effort involved in processing international invoices, converting foreign currencies into Indian Rupees (INR), applying relevant Indian Goods and Services Tax (GST) calculations, and generating downloadable PDF invoices.

In today's digital economy, Indian businesses, freelancers, software development companies, and agencies frequently purchase software subscriptions, cloud computing services, digital products, and professional services from international vendors. These vendors may issue invoices in currencies such as US Dollars (USD), British Pounds (GBP), Euros (EUR), Canadian Dollars (CAD), and Australian Dollars (AUD). Such invoices generally follow the tax and billing conventions of their respective countries and may not contain all the information required for Indian accounting and GST-related workflows.

InvoiceBridge addresses this challenge through an automated invoice-processing workflow. Users can upload an invoice in PDF or image format, extract relevant information using Optical Character Recognition (OCR), review and edit the extracted details, convert the monetary values into INR, and calculate GST based on the selected tax rate and supply type. The application then generates a downloadable PDF invoice containing the converted financial information and applicable GST breakdown.

The application uses a modern full-stack architecture comprising React, Vite, TypeScript, Tailwind CSS, and shadcn/ui for the frontend, along with Node.js, Express, and TypeScript for the backend. PostgreSQL is used for persistent data storage, managed through Drizzle ORM, while Clerk provides authentication. Tesseract.js and PDF.js support client-side invoice text extraction, and PDFKit is used for server-side PDF generation. An external exchange-rate API supports currency conversion, with caching and fallback rates to improve resilience.

The system also provides a dashboard for viewing invoice history, monitoring conversion status, reviewing summary statistics, downloading generated invoices, and reconverting previously processed invoices. Through these capabilities, InvoiceBridge aims to provide a streamlined and accessible solution for international invoice processing and Indian GST documentation.

The project also demonstrates the application of AI-assisted software development techniques, where AI tools can support requirements analysis, architecture planning, code generation, debugging, and iterative refinement. The development process can therefore be documented not only in terms of the final software product but also in terms of how prompts and AI-generated suggestions contribute to the software development lifecycle.

Problem Statement

Indian businesses and professionals regularly receive invoices from foreign service providers and suppliers. These invoices often contain financial information in foreign currencies and use tax structures that differ from the Indian GST system. Processing these invoices for Indian accounting and documentation can require manual data entry, currency conversion, tax interpretation, and invoice preparation.

The traditional manual process presents several challenges:

1. Manual data extraction: Invoice details such as vendor information, invoice numbers, dates, line items, quantities, and tax amounts must often be entered manually, increasing processing time and the possibility of human error.

2. Foreign currency conversion: Invoice amounts must be converted into INR using an appropriate exchange rate. Manually obtaining and applying exchange rates can introduce inconsistencies and calculation errors.

3. Different taxation systems: Foreign invoices may include systems such as US sales tax, UK and European VAT, Canadian GST/HST, or Australian GST. These tax structures must be considered when determining the appropriate taxable base for subsequent calculations.

4. Indian GST calculation requirements: Preparing an Indian GST-format invoice may require the selection of an applicable GST rate, identification of the place of supply, and calculation of CGST and SGST for intra-state supplies or IGST for inter-state supplies.

5. Invoice preparation and documentation: Manually preparing a formatted invoice and maintaining records of previously processed invoices can be time-consuming and difficult to manage consistently.

6. Lack of an integrated workflow: Invoice extraction, currency conversion, tax calculation, PDF generation, and invoice history management may require separate tools or manual processes, reducing operational efficiency.

Therefore, the problem addressed by Invoice Bridge is the absence of a single, streamlined workflow for extracting information from foreign invoices, converting currencies, applying configurable Indian GST calculations, and generating structured PDF invoices.

The proposed system integrates these operations into a web-based application. It enables users to upload invoices, review extracted data, modify relevant fields, calculate converted amounts and GST components, generate downloadable PDF documents, and manage invoice records through a centralized dashboard.

By combining document-processing technologies, currency-conversion services, tax-calculation logic, database management, and automated PDF generation, InvoiceBridge aims to reduce repetitive manual work, improve calculation consistency, and simplify international invoice processing for Indian businesses and professionals.

1.2 Existing Project and Limitations

Several online platforms provide invoice management, accounting, currency conversion, and tax calculation. However, converting foreign invoices into Indian GST-format invoices may require additional processing or configuration.

Sr. No.	Online Tool	Main Features	Limitations
1	Zoho Books	Accounting, GST invoicing, multi-currency support.	May require additional processing for foreign invoice conversion.
2	Xero	Cloud accounting and currency conversion.	Indian GST-specific workflows may require configuration.
3	TallyPrime	Accounting and GST management.	Automated foreign invoice extraction may require additional steps.
4	Zoho Invoice	Invoice creation and billing.	Primarily focuses on invoicing rather than foreign invoice conversion.
5	FreshBooks	Invoicing and expense tracking.	May require additional processing for Indian GST-format invoices.
6	Avalara	Tax automation and compliance solutions.	The required conversion workflow depends on the product and integration.

Table 1.2 (a): Comparison of Existing Online Tools

Limitations Addressed by InvoiceBridge

Sr. No.	Existing Limitation	InvoiceBridge Solution
1	Manual invoice data entry.	Extracts invoice text using OCR.
2	Manual currency conversion.	Converts foreign currency amounts into INR using exchange-rate data.
3	Complex foreign tax structures.	Applies country-specific tax normalization rules.
4	Manual GST calculations.	Calculates CGST, SGST, or IGST using configurable GST rates.
5	Manual invoice preparation.	Generates downloadable PDF invoices.
6	Difficulty tracking invoices.	Maintains invoice history and conversion status.
7	Multiple tools required.	Integrates extraction, conversion, GST calculation, and PDF generation.
8	Repeated processing.	Allows users to review, edit, and reconvert invoices.

Conclusion: InvoiceBridge integrates foreign invoice extraction, currency conversion, Indian GST calculations, PDF generation, and invoice management into a single web application, reducing repetitive manual work and simplifying invoice processing.

1.3 Objectives

The main objective of InvoiceBridge is to automate the conversion of foreign invoices into Indian GST-format invoices through an integrated web application.

The specific objectives are:

1. Automated Invoice Extraction: Extract invoice details from PDF and image files using OCR technology.
2. Currency Conversion: Convert foreign currency amounts into Indian Rupees (INR) using exchange-rate data.
3. GST Calculation: Calculate applicable GST components, including CGST, SGST, and IGST.
4. Tax Normalization: Handle foreign tax structures using country-specific conversion rules.
5. PDF Generation: Generate downloadable invoice PDFs containing converted amounts and GST details.
6. Invoice Management: Store invoice records and maintain conversion history using PostgreSQL.
7. User Authentication: Provide secure, authenticated access using Clerk.
8. Dashboard and Analytics: Display invoice statistics, conversion statuses, and processing history.
9. AI-Assisted Development: Use AI tools to support project planning, code generation, debugging, and iterative software development.

1.4 Proposed AI-Assisted Approach

The proposed approach uses Artificial Intelligence (AI) tools to support the development of InvoiceBridge. AI assistance is used to plan the application, generate code, resolve errors, and improve the implementation through iterative prompting.

The development approach consists of the following steps:

1. Requirement Analysis: Define the project problem, objectives, features, and technology requirements using AI assistance.
2. System Design: Plan the frontend, backend, database schema, API endpoints, and invoice-processing workflow.
3. Code Generation: Use AI-generated suggestions to develop UI components, backend APIs, OCR processing, GST calculations, and PDF generation.
4. Integration: Connect the frontend, backend, PostgreSQL database, authentication, and exchange-rate API.
5. Debugging and Testing: Use AI assistance to understand errors, refine code, and test the application's functionality.
6. Iterative Improvement: Refine prompts and modify the implementation based on testing results and project requirements.

This approach combines AI-assisted development with human review and testing. The generated code and suggestions must be evaluated before integration to ensure correctness, security, and reliability.

Note: The specific prompts and AI tools used at each development stage will be documented separately wherever the original history or project evidence is available.

2.Requirement Analysis

2.1 Functional and Non-Functional Requirements

The requirements of InvoiceBridge are divided into two categories: functional requirements and non-functional requirements. Functional requirements describe what the system should do, while non-functional requirements describe how the system should perform.

A. Functional Requirements

Functional requirements define the main features and operations provided by InvoiceBridge.

Sr. No.	Requirement	Description
1	User Authentication	The system allows users to sign in and access their invoice records through Clerk authentication.
2	Invoice Upload	Users can upload invoice documents in PDF, PNG, and JPG formats for processing.
3	Data Extraction	The system uses Tesseract.js and PDF.js to extract important details, such as invoice numbers, dates, vendor information, and amounts.
4	Currency Conversion	The application converts supported foreign currencies into Indian Rupees (INR) using exchange-rate data.
5	Tax Normalization	The system applies country-specific rules to handle foreign taxes before calculating Indian GST.
6	GST Calculation	Users can select a GST rate, and the system calculates CGST and SGST for intra-state supplies or IGST for inter-state supplies.
7	Invoice Editing	Users can review and correct extracted information, including invoice details, line items, and tax values.
8	PDF Generation	The system generates downloadable PDF invoices containing converted amounts and GST details.
9	Invoice History	The application stores invoice records and tracks their processing status in PostgreSQL.
10	Dashboard	Users can view invoice statistics, conversion history, total processed values, and monthly activity.
11	Invoice Reconversion	Users can revisit invoice details, make corrections, and process an invoice again.
12	Email Sharing	The application provides an email-sharing feature for sending generated invoices to recipients.

B. Non-Functional Requirements

Non-functional requirements describe the quality, security, usability, and reliability expected from InvoiceBridge.

Sr. No.	Requirement	Description
1	Performance	The system should process invoice information efficiently and display conversion results without unnecessary delays.
2	Security	Authentication and user-associated invoice records help prevent unauthorized access to invoice information.
3	Reliability	Exchange-rate caching and fallback values help the application handle temporary exchange-rate service issues.
4	Usability	The interface should be simple and easy to understand, allowing users to upload, review, convert, and download invoices easily.
5	Maintainability	TypeScript and a modular project structure make the code easier to understand, update, and debug.
6	Data Storage	PostgreSQL stores invoice details and conversion records for later access and management.
7	Scalability	The modular frontend and backend architecture supports future improvements and additional features.
8	Compatibility	The application should work with modern web browsers and support the configured invoice file formats.
9	Accuracy	Currency conversion and GST calculations should follow the configured rules, and users should be able to review the results before generating invoices.
10	Responsiveness	The user interface should adapt to different screen sizes, including desktops, tablets, and mobile devices.

Summary: These requirements guide the development of InvoiceBridge by defining its essential features and expected quality. Together, they help ensure that the application provides a convenient workflow for foreign invoice extraction, currency conversion, GST calculation, PDF generation, and invoice management.

2.2 User Stories and Acceptance Criteria

User stories describe the features of InvoiceBridge from the user's point of view. Acceptance criteria define the conditions that must be satisfied for each feature to work as expected.

Table 2.2: User Stories and Acceptance Criteria

Sr. No.	User Story	Acceptance Criteria
1	As a user, I want to sign in securely so that I can access my invoices.	The system authenticates users and restricts access to authorized invoice records.
2	As a user, I want to upload a foreign invoice so that I can convert it.	The system accepts supported PDF, PNG, and JPG files within the configured file-size limit.

3	As a user, I want invoice details extracted automatically so that I do not need to enter everything manually.	The system extracts available invoice information using OCR and displays it for review.
4	As a user, I want to correct extracted information so that errors can be fixed before conversion.	The system allows users to edit invoice details and line items.
5	As a user, I want to convert foreign currency into INR so that I can view the invoice value in Indian currency.	The system applies an available exchange rate and displays the converted amount.
6	As a user, I want GST calculated automatically so that I can view the applicable tax breakdown.	The system calculates the selected GST rate and displays CGST/SGST or IGST according to the selected supply type.
7	As a user, I want a downloadable PDF invoice so that I can save the converted document.	The system generates a PDF containing the converted invoice details and tax calculations.
8	As a user, I want to view my invoice history so that I can access previously processed invoices.	The dashboard displays stored invoice records and their processing statuses.
9	As a user, I want to reconvert an invoice so that I can correct details and generate an updated document.	The system allows an invoice to be reopened, edited, and processed again.
10	As a user, I want to view conversion statistics so that I can understand my invoice-processing activity.	The dashboard displays available invoice counts, processed values, and monthly statistics.
11	As a user, I want to share a generated invoice by email so that I can send it to another person.	The system provides an email-sharing workflow for the generated invoice.

Acceptance Criteria Summary

The main acceptance criteria for InvoiceBridge are:

- Users can authenticate and access their own invoice records.
- Supported invoice files can be uploaded and processed.
- Extracted information can be reviewed and corrected before conversion.
- Currency conversion and configured GST calculations produce the expected results for valid inputs.
- Converted invoices can be generated as downloadable PDFs.
- Invoice history, processing status, and dashboard statistics are available.
- Users can access reconversion and email-sharing features.

These criteria provide a practical basis for checking whether the application's main features work as intended. Actual GST treatment should still be verified for the specific transaction.

2.3 Prompts, AI Outputs and Refinement

Artificial Intelligence (AI) tools were used as development assistants during the development of InvoiceBridge. They helped in planning the project, selecting technologies, designing the user interface, generating code, solving errors, and improving different features.

The development process involved giving instructions to AI, reviewing its responses, and refining the instructions whenever the initial output did not fully satisfy the requirements.

Note: The prompts and outputs below are reconstructed examples based on the documented project implementation. They are not verified copies of the original conversation history.

1. Initial Project Idea

Prompt 1: Initial Prompt

"i want to make website for convert foreign invoice into indian gst invoice. user upload invoice and website convert it in INR and calculate GST. help me how to create this project."

AI Output

Great idea! Invoice conversion is a useful project because many Indian businesses purchase software, cloud services, and other products from international vendors. Their invoices may be in foreign currencies and follow different tax systems.

To build this project, we can divide the application into the following main features:

- Invoice Upload: Allow users to upload their foreign invoices.
- Invoice Data Extraction: Extract important details such as invoice number, date, vendor name, and amount.
- Currency Conversion: Convert foreign currency values into Indian Rupees (INR).
- GST Calculation: Calculate the required GST components based on the selected rate and supply type.
- PDF Generation: Generate a downloadable invoice after conversion.
- Invoice History: Allow users to view previously processed invoices.

A simple workflow would be:

Upload Invoice

Extract Invoice Data

Convert Currency and Calculate GST

Generate PDF Invoice

This is a good starting point. We can develop each feature separately and integrate them into one complete web application.

Refinement

The initial response provided a basic project idea, but I wanted to build a complete SaaS application rather than a simple invoice converter. Therefore, the requirements were expanded to include authentication, database storage, OCR, a dashboard, and invoice management.

Refined Prompt:
"I want to develop a complete SaaS project named InvoiceBridge for converting foreign invoices into Indian GST invoices. Explain all required features, frontend, backend, database, APIs, and complete workflow step by step."

Improved AI Output
Excellent! Now we can plan InvoiceBridge as a complete full-stack SaaS application instead of a basic conversion tool.
The application can be divided into five major modules:

1. Frontend: Home page, upload page, conversion form, download page, and dashboard.
2. Backend: APIs for invoice processing, currency conversion, and PDF generation.
3. Database: Store invoice details, extracted information, and conversion records.
4. Processing Engine: Handle OCR, foreign tax normalization, currency conversion, and GST calculations.
5. Invoice Management: Provide authentication, invoice history, PDF downloads, and reconversion.

This modular approach makes the project easier to develop, test, and maintain. It also provides a clear development roadmap for implementing each feature step by step.

2. Technology Stack Selection
Prompt 2: Initial Prompt
"which technology should i use for this project. i want modern frontend backend and database."

AI Output
Good question! Selecting the right technology stack is important because InvoiceBridge requires document processing, currency conversion, database storage, authentication, and PDF generation.
There are several possible choices. React can be used for the frontend, Node.js or Django for the backend, and PostgreSQL or MySQL for the database. However, selecting technologies independently may create integration difficulties later.
For this project, we should select technologies that work well together and support a modular architecture.

Refinement
The initial response provided several alternatives but did not clearly select one stack for the actual requirements.

Refined Prompt:

"Select one suitable technology stack for InvoiceBridge. It should support OCR, currency conversion, GST calculation, PDF generation, user authentication, and invoice history. Explain the purpose of each technology."

Improved AI Output

Great! For InvoiceBridge, a TypeScript-based full-stack architecture is a suitable choice because it allows typed code across the frontend and backend.

The following stack matches the documented implementation:

Component	Technology	Purpose
Frontend	React, Vite, TypeScript	Build the user interface
Styling	Tailwind CSS, shadcn/ui	Create responsive UI components
Backend	Node.js, Express 5, TypeScript	Handle APIs and business logic
Database	PostgreSQL	Store invoice records
ORM	Drizzle ORM	Manage database operations
Authentication	Clerk	Handle user authentication
OCR	Tesseract.js, PDF.js	Extract invoice text
PDF Generation	PDFKit	Generate invoice PDFs
Exchange Rates	Exchange-rate API	Obtain currency conversion rates
Email	Resend	Support email sharing

Using this stack provides a clear separation between the user interface, backend processing, and persistent data storage. Each module can be developed and tested separately before integration.

3. Frontend Development

Prompt 3: Initial Prompt

"make frontend for invoice converter website and make it beautiful."

AI Output

Sure! Let's create a clean and modern interface for the invoice converter.

A basic version could contain an application title, a file upload button, and a convert button.

Invoice Converter

Upload Your Invoice

[Choose File]

[Convert Invoice]

This is a good starting point for testing the upload feature. However, a single page would not provide enough functionality for a complete SaaS application with invoice history and conversion management.

Refinement
I needed a more complete interface with separate pages and a consistent design.

Refined Prompt:
"Create a modern and responsive SaaS frontend for InvoiceBridge. Include Home, Upload, Convert, Download, and Dashboard pages. Use React, TypeScript, Tailwind CSS, and shadcn/ui. Keep the design clean and easy to use."

Improved AI Output
Nice! We can organize the frontend into separate pages so users can understand the workflow easily.

- Home Page: Introduces InvoiceBridge and explains its main benefits.
- Upload Page: Allows users to select and upload invoice files.
- Convert Page: Displays extracted information and editable fields for currency and GST calculations.
- Download Page: Shows the conversion summary and provides the generated PDF.
- Dashboard: Displays invoice statistics, processing statuses, and invoice history.

React provides reusable components, while Tailwind CSS and shadcn/ui support a consistent interface. This structure also makes it easier to add validation messages, loading indicators, and error handling.

4. Invoice Upload and OCR
Prompt 4: Initial Prompt
"how to read data from uploaded invoice automatically."

AI Output
You can use OCR (Optical Character Recognition) to extract text from an invoice.
For example, OCR can help identify information such as the invoice number, vendor name, invoice date, and total amount.
A basic process is:

1. Upload the invoice.
2. Extract the text.
3. Identify important information.
4. Display the extracted data.

However, OCR does not always understand the meaning of every field correctly. A scanned document may also produce incomplete or incorrect text.
Refinement
I wanted the application to support both PDF and image invoices, extract useful information, and allow users to correct mistakes.

Refined Prompt:
"Implement client-side OCR for PDF, PNG, and JPG invoices using Tesseract.js and PDF.js. Extract invoice number, date, vendor name, amounts, and tax details. Display the extracted information in an editable form so the user can correct mistakes."

Improved AI Output
Great! We can use PDF.js to process PDF documents and Tesseract.js to recognize text from image content.
The workflow can be implemented as follows:

1. The user uploads an invoice.
2. The application processes the document in the browser.
3. OCR extracts the available text.
4. The extracted text is examined to identify invoice fields.
5. The application displays the information in an editable form.

The user can review the extracted information before continuing. This is important because OCR may misread numbers, dates, or vendor details, and incorrect data could affect subsequent calculations.

5. Currency Conversion
Prompt 5: Initial Prompt
"convert USD to INR and also other currencies."

AI Output
Currency conversion can be performed by multiplying the foreign amount by the exchange rate.
For example, if an invoice contains 100 USD and the exchange rate is ₹83 per USD:
\[100 \times 83 = ₹8,300 \]
This calculation is simple, but the application also needs to obtain exchange-rate information and handle situations where the external API does not respond.

Refinement
Refined Prompt:
"Implement currency conversion for USD, GBP, EUR, CAD, and AUD into INR using an exchange-rate API. Add caching to reduce repeated requests and fallback rates when the API is unavailable. Allow users to adjust the exchange rate manually."
Improved AI Output
Good! The currency conversion module should support multiple currencies and avoid depending entirely on a single API request.
The implementation can:

- Retrieve exchange rates from an external service.
- Convert supported currencies into INR.
- Cache exchange-rate information for one hour.
- Use configured fallback rates when the service is unavailable.
- Allow manual adjustment of the exchange rate in the conversion form.

Caching helps reduce repeated API requests, while fallback values help the workflow continue during temporary service failures. However, fallback rates may not reflect current market rates, so the applied rate should remain visible to the user.

6. GST Calculation and Tax Normalization

Prompt 6: Initial Prompt

"calculate GST on invoice amount and show CGST SGST IGST."

AI Output

GST can be calculated using the taxable amount and selected GST rate.

For example, if the taxable amount is ₹10,000 and the selected rate is 18%, the GST amount is ₹1,800.

The tax components depend on the selected supply type. However, foreign invoices may already contain VAT, sales tax, or other taxes, so simply adding Indian GST to the total foreign invoice amount may produce an incorrect result.

Refinement

Refined Prompt:

"Create a configurable GST calculation module for InvoiceBridge. Support GST rates of 0%, 5%, 12%, 18%, and 28%. Calculate CGST and SGST for intra-state supplies and IGST for inter-state supplies. Also handle foreign VAT, sales tax, and GST/HST using country-specific normalization rules."

Improved AI Output

Excellent! The tax calculation module should separate foreign tax normalization from Indian GST calculation.

The workflow should:

1. Apply the configured normalization rule for the source country's tax system.
2. Convert the relevant amount into INR.
3. Apply the selected GST rate.
4. Calculate CGST and SGST for intra-state supplies.
5. Calculate IGST for inter-state supplies.
6. Display the taxable amount, GST breakdown, and final amount.

The conversion form should show the calculations clearly so users can review them before generating the invoice. The appropriate tax treatment must still be verified for each transaction under the applicable Indian rules.

7. Database Design

Prompt 7: Initial Prompt

"create database for storing uploaded invoices and converted invoices."

AI Output

We can start with a database table containing the invoice ID, filename, original amount, currency, and converted amount.

For a small demonstration, this structure may be enough to store basic information. However, InvoiceBridge needs to track the original invoice separately from its conversion result.

Refinement

Refined Prompt:

"Design a PostgreSQL database for InvoiceBridge using Drizzle ORM. Separate uploaded invoices and converted invoice records. Store user ID, invoice metadata, extracted data, conversion status, exchange rate, GST breakdown, and generated PDF path."

Improved AI Output

Good approach! Separating the uploaded invoice from its converted record makes it easier to maintain processing history and link a conversion to its original document.

The documented database design uses two main tables:

Table	Purpose
invoices	Stores user association, file information, extracted data, source currency, and processing status
converted_invoices	Stores converted INR amounts, GST components, exchange rate, converted data, and PDF path

PostgreSQL stores the records, and Drizzle ORM manages database operations. A relationship between the tables connects conversion results with the corresponding original invoice.

8. PDF Generation

Prompt 8: Initial Prompt

"generate PDF invoice after conversion."

AI Output

We can generate a PDF containing the invoice details and the final converted amount.

However, a PDF containing only the final amount would not be very useful for accounting or reviewing the calculation. The document should include the important invoice information and the tax breakdown.

Refinement
Refined Prompt:
"Generate a structured PDF invoice using PDFKit. Include vendor and buyer details, invoice number, date, line items, taxable amount, GST breakdown, and final amount in INR. Provide a download option after successful conversion."

Improved AI Output
Great! PDFKit can be used on the backend to generate a structured invoice document.
The PDF generation process should include:

- Vendor and buyer information.
- Invoice number and date.
- Item descriptions and amounts.
- Taxable amount and GST breakdown.
- Final invoice amount in INR.

After successful generation, the user should be able to download the PDF from the application. The documented implementation also uses a Base64 JSON response for the download endpoint to avoid binary-stream corruption in the configured proxy workflow.

9. Authentication and Invoice Security
Prompt 9: Initial Prompt
"add login system to my website."

AI Output
A login system allows users to sign in before accessing the application.
However, adding a login page alone is not enough. The backend must also check which user is requesting an invoice and whether that user is allowed to access it.

Refinement
Refined Prompt:
"Integrate Clerk authentication with InvoiceBridge. Associate invoices with user IDs and ensure backend routes verify authentication and restrict invoice operations to the correct user."

Improved AI Output
Good! Clerk can handle user authentication, while the backend can associate invoice records with the authenticated user's identifier.
The backend should verify the user before allowing invoice access, downloads, edits, reconversion, or deletion. These checks help prevent users from accessing another user's invoice records.

The implementation must be tested to confirm that the access restrictions are enforced on the backend, rather than relying only on hiding interface elements.

10. Dashboard and Invoice History
Prompt 10: Initial Prompt
"make dashboard for showing invoices."

AI Output
The dashboard can show a list of uploaded invoices, their amounts, and their current statuses. This is a useful beginning, but it does not provide much information about the overall processing activity of the user.

Refinement
Refined Prompt:
"Create an InvoiceBridge dashboard showing total uploaded invoices, successfully converted invoices, total INR value, monthly conversion activity, conversion success rate, and invoice history. Include download, reconversion, and delete actions where applicable."

Improved AI Output
Nice! A dashboard with summary statistics can help users understand their invoice-processing activity without opening every record separately.
The dashboard should display:

- Total uploaded invoices.
- Successfully converted invoices.
- Total processed value in INR.
- Monthly conversion activity.
- Conversion success rate.
- Invoice history and processing status.

Users can also access the available actions for downloading, reconverting, or deleting invoices. This makes invoice management more convenient and organized.

11. Debugging and Error Handling
Prompt 11: Initial Prompt
"my code is not working fix it."

AI Output
I can help you debug the issue, but I need more information about the error.
Please provide the error message, the relevant code, and the steps that cause the problem. Without this information, it would be difficult to identify the exact cause.

Refinement

Refined Prompt:

"Analyze the following error message and related code. Identify the root cause, explain why it occurs, and suggest the smallest safe code change. Preserve existing features and provide steps to verify the fix."

Improved AI Output

Great! A more specific debugging prompt makes it easier to identify the actual problem instead of guessing.

The debugging process should include:

1. Understanding the error message.
2. Identifying the relevant file or function.
3. Explaining the root cause.
4. Suggesting a minimal code change.
5. Testing the fix and checking that existing features still work.

This approach helps make debugging more systematic and reduces the chance of introducing unrelated problems while fixing an error.

12. Final Review and Improvement

Prompt 12: Final Prompt

"Review the complete InvoiceBridge project and identify missing features, possible bugs, security issues, and improvements in the invoice conversion workflow."

AI Output

Excellent! Reviewing the complete project is an important step before considering the application ready.

The review should cover the following areas:

- OCR: Check whether extracted invoice details can be reviewed and corrected.
- Currency Conversion: Check exchange-rate retrieval, caching, and fallback behaviour.
- GST Calculations: Verify tax calculations, rounding, and the selected supply type.
- PDF Generation: Confirm that generated invoices contain the expected information and can be downloaded.
- Security: Verify authentication and user-specific access controls.
- Database: Check that invoice records and conversion results remain consistent.
- Error Handling: Test invalid files, missing fields, and external service failures.
- Workflow Testing: Test the complete process from upload to PDF download.

Refinement

The final review should not rely only on AI suggestions. The identified issues must be checked against the source code and actual application behaviour. Any changes should be tested to confirm that the expected features continue to work.

Conclusion

The examples demonstrate how AI-assisted development can progress from a general project idea to detailed implementation requirements. Initial prompts provide a starting point, while refined prompts specify the expected behaviour, technical constraints, and error-handling requirements.

AI tools can help generate code, explain errors, and improve development efficiency. However, the developer remains responsible for reviewing the generated output, integrating the modules, and testing the application.

3. System Design

3.1 System Architecture and Module Design

InvoiceBridge follows a client-server architecture that connects the frontend, backend, database, and external services to convert foreign invoices into Indian GST-compliant invoices. The system is designed to make invoice processing easier by combining invoice upload, data extraction, currency conversion, GST calculation, and PDF generation in a single application.

3.1.1 System Architecture

The system consists of the following main components:

1. Frontend Module: Developed using React, Vite, TypeScript, and Tailwind CSS. It provides the user interface for uploading invoices, reviewing extracted information, editing invoice details, configuring GST, and downloading converted invoices.
2. Backend Module: Developed using Node.js, Express 5, and TypeScript. It handles API requests, validates uploaded invoice data, manages the conversion workflow, and connects the frontend with the database and external services.
3. Invoice Processing Module: Extracts invoice information from uploaded PDF and image files using Tesseract.js and PDF.js. The extracted information is presented to users for review and correction before conversion.
4. Currency Conversion and GST Module: Converts foreign currency amounts into Indian Rupees using exchange-rate data. It also normalizes foreign tax information and calculates applicable Indian GST, including CGST, SGST, or IGST based on the selected tax configuration.
5. Database Module: Uses PostgreSQL with Drizzle ORM to store invoice records, extracted information, conversion results, tax details, and processing status.
6. PDF Generation Module: Uses PDFKit to generate the final GST invoice in PDF format. The generated invoice can be downloaded and, where required, shared through email.
7. Authentication and External Services: Clerk manages user authentication, an exchange-rate API supplies currency conversion data, and Resend supports invoice email delivery.

3.1.2 Module Design

The system is divided into separate modules to improve maintainability, readability, and future scalability. Each module performs a specific function and communicates with other modules through defined interfaces and API endpoints.

Module	Main Responsibility
User Authentication	Handles user registration, sign-in, and protected access.
Invoice Upload	Accepts supported PDF and image files.
OCR and Data Extraction	Extracts invoice text and details from uploaded files.

Invoice Review and Editing	Allows users to check and correct extracted information.
Currency Conversion	Converts foreign currency values into INR.
GST Calculation	Calculates applicable Indian GST and tax components.
Invoice Management	Stores and manages original and converted invoice records.
PDF Generation	Generates the final GST invoice in PDF format.
Dashboard and History	Displays invoice records, processing statistics, and available actions.
Email Sharing	Supports sending generated invoices through email.

3.1.3 System Architecture Diagram

The following diagram illustrates the interaction between users, the frontend, backend processing modules, external services, database, and file storage.

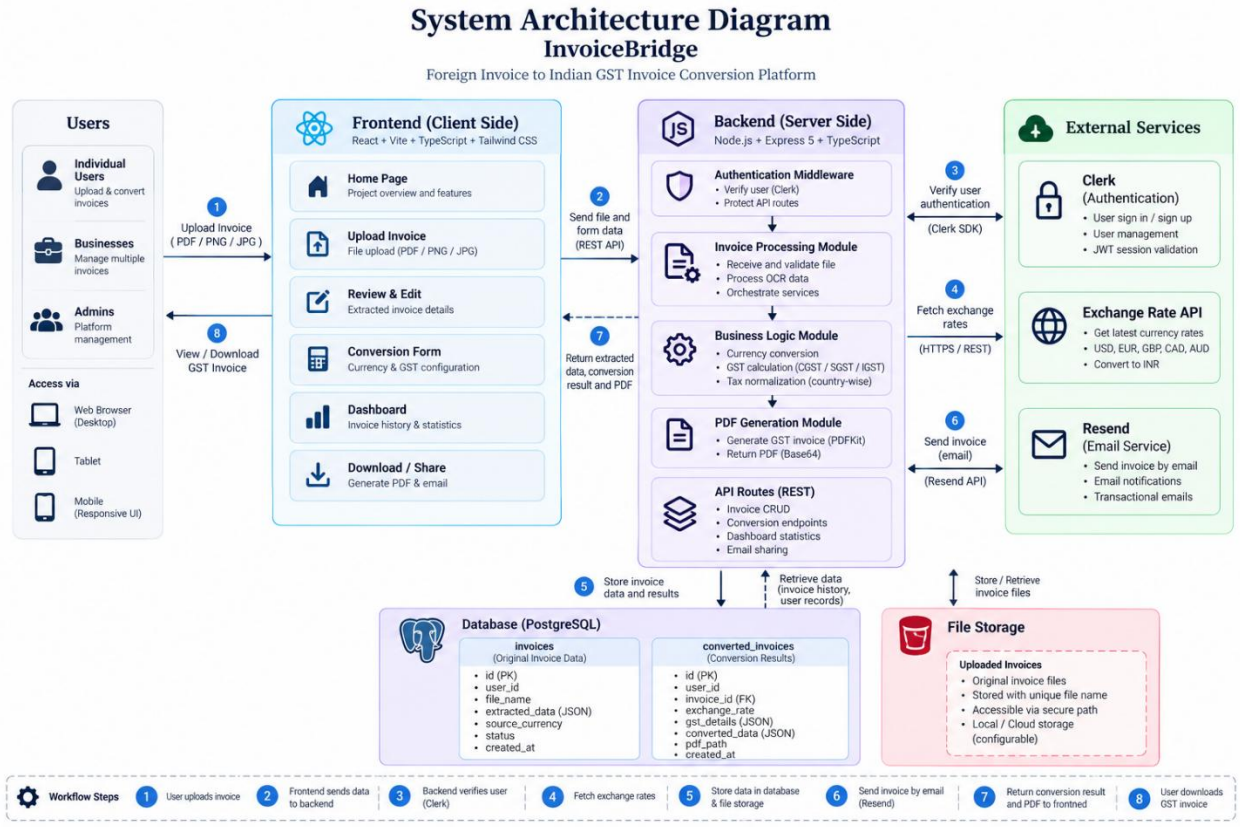


Figure 3.1: System Architecture Diagram of InvoiceBridge

3.1.4 Working of the System

The system follows a sequence of steps to convert a foreign invoice into an Indian GST invoice:

1. The user signs in and uploads a foreign invoice in PDF or image format.

2. The frontend processes the document using browser-based OCR tools and displays the extracted information for review.
3. The user checks the extracted details and corrects any inaccurate information.
4. The backend obtains exchange-rate data and converts the invoice amount into Indian Rupees.
5. The GST module applies the selected GST rate and calculates the appropriate tax components.
6. The system stores the invoice details and conversion results in the database.
7. The PDF generation module creates the final GST invoice.
8. The user can download the converted invoice, view it in the dashboard, or share it through email.

This modular architecture provides a structured workflow for invoice conversion and helps keep document extraction, business logic, data storage, and output generation as separate responsibilities.

3.2 User Interface Design / Wireframes

The user interface of InvoiceBridge is designed to provide a simple, modern, and user-friendly experience. It uses a clean layout, clear navigation, blue and purple accent colours, and responsive components to make invoice conversion easy for users. The interface consists of three main screens: the home page, invoice upload page, and dashboard.

3.2.1 Home Page

The home page introduces InvoiceBridge and explains its main purpose: converting foreign invoices into Indian GST format. It highlights support for invoices from the United States, United Kingdom, European Union, Canada, and Australia. The page also provides options to get started or sign in.

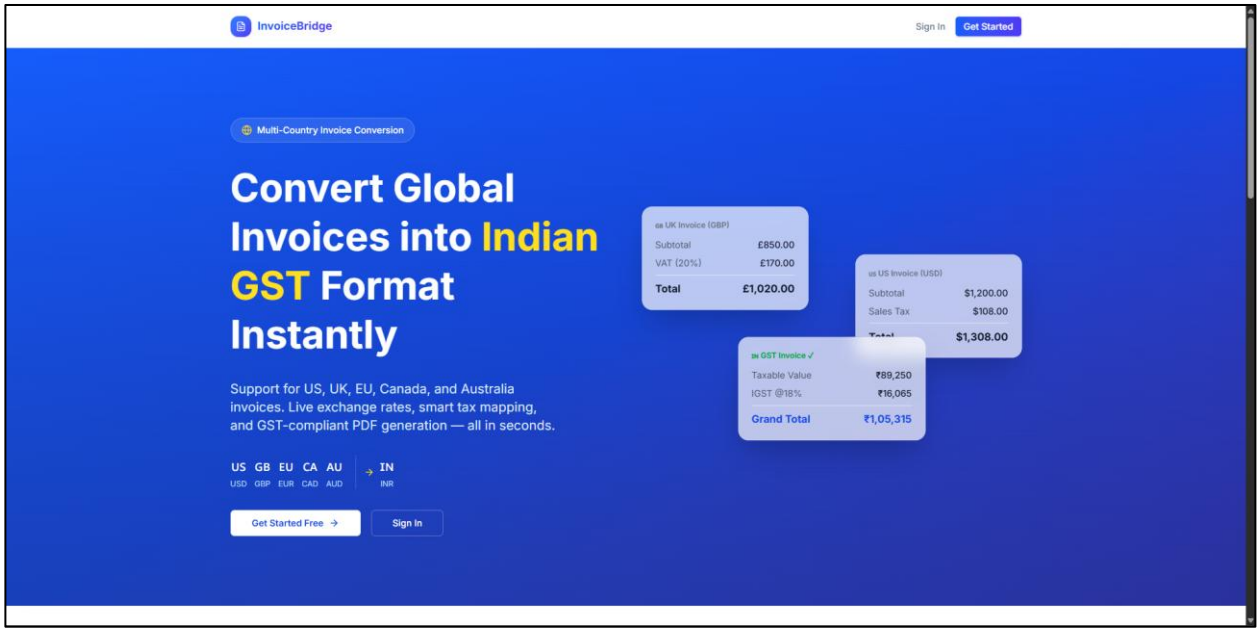


Figure 3.2: InvoiceBridge Home Page

3.2.2 Invoice Upload Page

The invoice upload page allows users to select the country from which the invoice originated and upload a document in PDF, PNG, or JPG format. Users can drag and drop a file or browse their device to select one. The page provides an Upload & Extract button to begin invoice processing and text extraction.

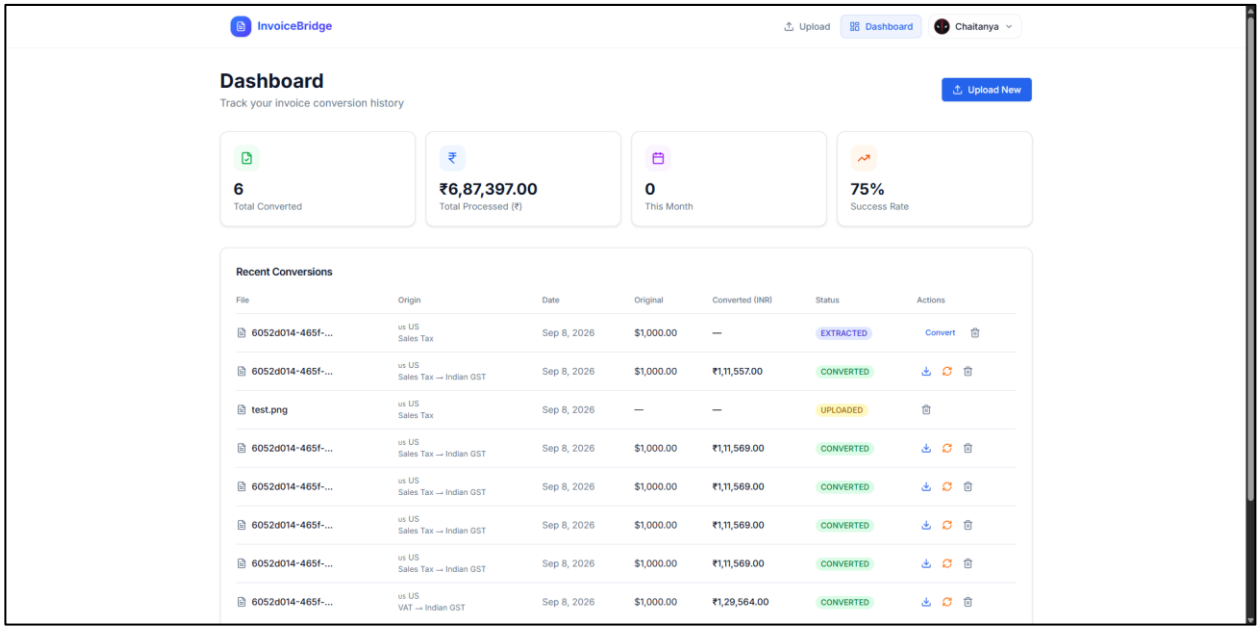
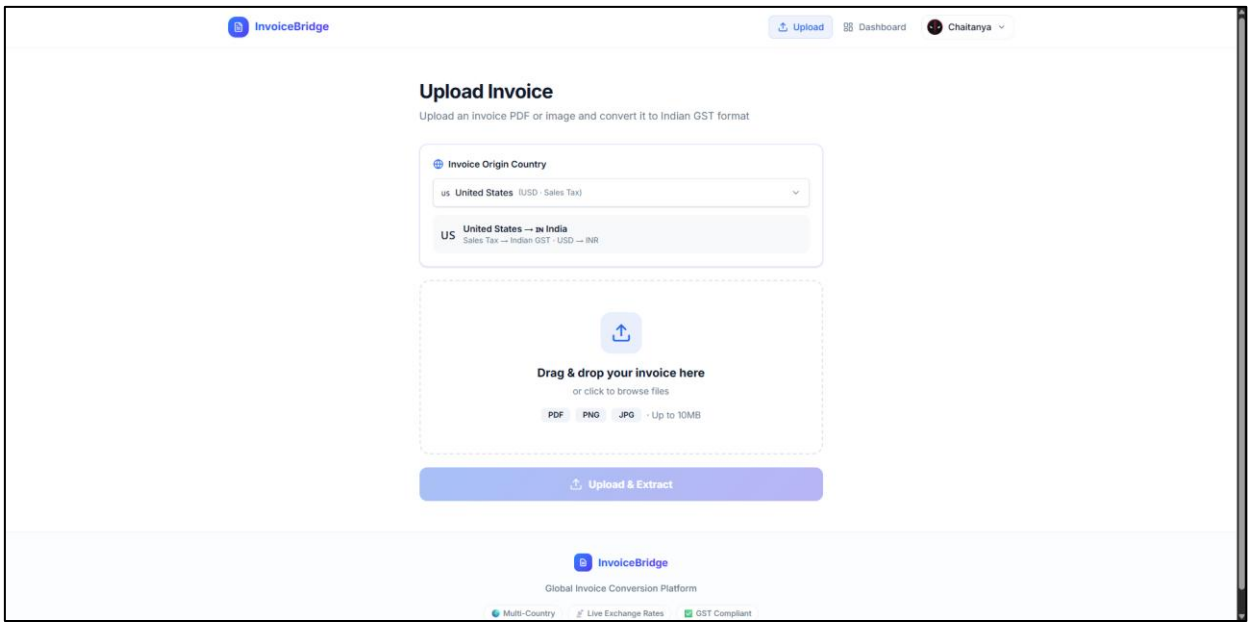


Figure 3.3: Invoice Upload Interface

3.2.3 Dashboard

The dashboard provides an overview of the user's invoice conversion activities. It displays key statistics, including total converted invoices, total processed amount in Indian Rupees, monthly conversions, and success rate. A recent conversions table shows invoice details, country of origin, date, original amount, converted amount, processing status, and available



actions. Users can download converted invoices, reconvert invoices, delete records, or upload a new invoice.

3.2.4 UI Design Principles

The interface follows these main design principles:

- Simplicity: Clear labels and straightforward navigation help users complete tasks easily.
- Consistency: Similar colours, buttons, cards, and layouts are used throughout the application.
- Feedback: Status labels help users identify whether an invoice is uploaded, extracted, or converted.
- Accessibility: Readable text, recognisable icons, and clearly separated sections improve usability.
- Responsiveness: The interface is designed to adapt to different screen sizes.

Overall, the user interface supports the complete invoice conversion workflow, from uploading a foreign invoice to viewing and downloading the converted Indian GST invoice.

3.3 Design Prompts, AI Outputs and Final Design

The user interface of InvoiceBridge was designed and refined with the assistance of Google Antigravity, an AI-powered development environment. AI-assisted development helped translate project requirements into practical UI components and improve the overall appearance and usability of the application.

3.3.1 Initial Design Prompt

Prompt:

"Create a modern website for my InvoiceBridge project. It converts foreign invoices into Indian GST invoices. I need a home page, invoice upload page and dashboard. Make the website simple, attractive and easy to use."

AI Output (Antigravity):

Antigravity helped establish the initial page structure, including a landing page, invoice upload interface, and dashboard. This design provided a foundation for organising the main features and navigation of InvoiceBridge.

3.3.2 Prompt for Modern UI Design

Prompt:

"Make the UI more professional and modern. Use a blue and purple theme, clean typography, proper spacing, rounded cards and simple icons. It should look like a real SaaS product and all pages should follow the same design."

AI Output (Antigravity):

The interface was refined with consistent colours, improved spacing, reusable components, and a clear visual hierarchy. The updated design made the application more organised and maintained a consistent appearance across different pages.

3.3.3 Prompt for Invoice Upload Interface

Prompt:

"Create an invoice upload page where users can select the invoice origin country like USA, UK, EU, Canada and Australia. Add a drag and drop area for PDF, PNG and JPG files. Include an Upload and Extract button and show the supported file formats clearly."

AI Output (Antigravity):

Antigravity proposed an upload interface with a country selection field, a drag-and-drop upload area, supported file-format labels, and an upload-and-extract button. The layout guides users through the initial step of invoice conversion.

3.3.4 Prompt for Dashboard Design

Prompt:

"Design a clean dashboard for InvoiceBridge. Show total converted invoices, total processed amount in INR, this month's conversions and success rate. Below these cards, show recent conversions with file name, origin, date, original amount, converted amount, status and actions for download, reconvert and delete."

AI Output (Antigravity):

The dashboard was designed with summary cards and a structured invoice history table. The cards provide an overview of conversion activity, while status indicators and action icons help users manage individual invoice records.

3.3.5 Final Design

The final interface consists of three main screens:

- Home Page: Introduces InvoiceBridge, explains its multi-country invoice conversion features, and provides sign-in and get-started options.
- Invoice Upload Page: Allows users to select the invoice's origin country and upload supported documents for extraction.
- Dashboard: Displays conversion statistics, invoice history, processing status, and actions for managing invoices.

The implementation uses React, TypeScript, Vite, Tailwind CSS, and reusable UI components to maintain a consistent design throughout the application.

3.3.6 Outcome

AI-assisted design helped accelerate user interface development and provided a structured approach to refining the application's appearance. The final design focuses on usability,

consistency, and straightforward navigation, allowing users to upload foreign invoices, review conversion results, and download Indian GST invoices.

The implemented home page, invoice upload page, and dashboard are presented in Section 3.2 as screenshots of the final interface.

4.Coding and Implementation

4.1 Development Environment and Technology Stack

InvoiceBridge was developed as a full-stack web application using modern technologies for frontend development, backend processing, database management, and document generation. The main objective was to build a system that can extract information from foreign invoices, convert amounts into Indian Rupees, calculate applicable GST, and generate a downloadable GST invoice.

The project follows a monorepo architecture using pnpm workspaces, which keeps the frontend, backend, and shared libraries organised within a single codebase. The application was deployed on Vercel, with PostgreSQL hosted on Neon for persistent data storage.

4.1.1 Development Environment

Tool	Purpose
Node.js 24	Runtime for server-side JavaScript and TypeScript
pnpm Workspaces	Managing packages and the monorepo structure
TypeScript	Type-safe development
Vite	Frontend development server and build tool
PostgreSQL	Storing invoice and conversion records
Drizzle ORM	Database schema management and queries
Git and GitHub	Source code version control
Google Antigravity	AI-assisted coding, debugging, and refinement
Vercel	Application deployment and hosting

4.1.2 Technology Stack

The application uses the following technologies for its main components.

Component	Technology	Purpose
Frontend	React 18	Building interactive user interfaces
Frontend tooling	Vite	Development and production builds

Language	TypeScript	Type safety and maintainability
Styling	Tailwind CSS	Responsive interface styling
UI components	shadcn/ui	Reusable interface components
Animations	Framer Motion	Interface animations
Routing	Wouter	Frontend navigation
Backend	Node.js 24	Server-side execution
Backend framework	Express 5	Handling API requests
Database	PostgreSQL	Persistent data storage
ORM	Drizzle ORM	Database operations
Authentication	Clerk	User authentication
OCR	Tesseract.js	Extracting text from invoice images
PDF processing	PDF.js	Reading PDF documents
PDF generation	PDFKit	Generating GST invoice PDFs
API specification	OpenAPI 3.1	Documenting API endpoints
API client	React Query and Orval	API data management and client-hook generation
Exchange-rate service	Exchange-rate API	Obtaining currency conversion rates
Email service	Resend	Sending invoices through email

4.1.3 Database and Storage Technologies

InvoiceBridge uses PostgreSQL, hosted on Neon, to store application data. Drizzle ORM provides a structured way to define database tables and perform database operations.

The system primarily uses two tables:

- invoices: Stores uploaded invoice details, file information, extracted data, source currency, processing status, and timestamps.
- converted_invoices: Stores conversion results, INR amounts, exchange rates, GST amounts, converted invoice data, and references to generated PDF files.

The application also maintains invoice-related files separately from database records. This separation allows the system to manage document files and structured invoice information independently.

4.1.4 Document Processing Technologies

The document processing workflow combines browser-based extraction with server-side processing.

1. Invoice upload: Users upload PDF or image files through the frontend.
2. Text extraction: Tesseract.js extracts text from images, while PDF.js supports PDF processing.
3. Invoice review: Extracted information is displayed so users can correct inaccurate fields.
4. Currency conversion: Foreign currency amounts are converted into INR using exchange-rate data.
5. GST calculation: The system calculates applicable GST components according to the selected tax configuration.
6. PDF generation: PDFKit generates the final converted invoice for download.

4.1.5 Deployment Environment

InvoiceBridge was deployed on Vercel to make the application accessible through a web interface. Neon provides the PostgreSQL database, while Clerk, the exchange-rate API, and Resend support authentication, currency conversion, and email functionality.

The deployment configuration includes the application build, required environment variables, database connectivity, and external service credentials. The frontend and backend must be configured according to their respective deployment requirements.

4.1.6 Summary

The selected technology stack provides a structured foundation for developing and maintaining InvoiceBridge. React and TypeScript support the frontend, Express handles backend operations, PostgreSQL stores invoice information, and specialised libraries support OCR and PDF generation. Deployment on Vercel makes the application accessible online, while Neon and external services provide the supporting infrastructure required for invoice conversion.

4.2 AI-Assisted Code Generation

AI-assisted code generation was used during the development of InvoiceBridge to generate initial code, implement features, and improve development efficiency. Google Antigravity

assisted with frontend components, backend API logic, database operations, invoice processing, and debugging.
The generated code was reviewed and modified according to the project requirements before integration.

4.2.1 Invoice Conversion

Prompt:

"Create a TypeScript function for InvoiceBridge that converts foreign invoice amounts into INR and calculates GST. If the supply is within the same state, calculate CGST and SGST. Otherwise calculate IGST."

AI Output (Antigravity):

The suggested approach was to convert the foreign amount into INR and calculate GST based on the selected rate and supply type.

Code Example:

```
type GstResult = {
  inrAmount: number;
  cgst: number;
  sgst: number;
  igst: number;
  total: number;
};
function calculateInvoice(
  amount: number,
  exchangeRate: number,
  gstRate: number,
  sameState: boolean
): GstResult {
  const inrAmount = amount * exchangeRate;
  const gst = inrAmount * (gstRate / 100);
  const cgst = sameState ? gst / 2 : 0;
  const sgst = sameState ? gst / 2 : 0;
  const igst = sameState ? 0 : gst;
  return {
    inrAmount,
    cgst,
    sgst,
    igst,
    total: inrAmount + gst
```

```
  };
}
```
Solution and Manual Modification: The example demonstrates the basic calculation logic. The complete application must validate amounts, exchange rates, and GST configuration before processing an invoice.

4.2.2 Invoice Status Management

Prompt:

"Create invoice statuses for my project to track an invoice from uploading to successful conversion, including failed processing."

AI Output (Antigravity):

An enum-based status structure helps maintain consistent status values throughout the application.

Code Example:

```
enum InvoiceStatus {
  UPLOADED = "UPLOADED",
  EXTRACTING = "EXTRACTING",
  EXTRACTED = "EXTRACTED",
  CONVERTING = "CONVERTING",
  CONVERTED = "CONVERTED",
  FAILED = "FAILED"
}
```

Solution and Manual Modification: These statuses correspond to the documented InvoiceBridge processing workflow. They help the frontend display invoice progress and the backend track processing results.

4.3 Debugging, Refactoring and Code Optimization

Debugging and refactoring were important during InvoiceBridge development. AI assistance helped analyse potential errors, suggest solutions, and improve code organisation. Suggested changes were reviewed before being applied.

4.3.1 PDF Download Handling

Problem: The application needed to transfer generated PDF invoices from the backend to the frontend without binary response corruption.

Prompt:

"My PDF invoice is generated on the backend, but I am facing issues while downloading it from the frontend. Suggest a safe way to return the PDF through the API."

AI Output (Antigravity):
One possible solution was to encode the generated PDF as Base64 and return it inside a JSON response.
Code Example:

```
import fs from "node:fs/promises";
async function createPdfResponse(
  pdfPath: string
) {
  const buffer = await fs.readFile(pdfPath);
  return {
    pdfBase64: buffer.toString("base64"),
    fileName: "converted-invoice.pdf",
    contentType: "application/pdf"
  };
}
```

Solution: The documented InvoiceBridge download workflow uses a Base64-encoded PDF in a JSON response from /api/invoices/:id/download. The route must also check user permissions and handle file errors.

4.3.2 Currency Conversion Reliability
Problem: An exchange-rate API may be unavailable, which can interrupt currency conversion.
Prompt:
"What if the exchange rate API is not working? I want InvoiceBridge to handle this problem instead of failing immediately."

AI Output (Antigravity):
The suggested approach was to use caching and fallback exchange rates when live rate retrieval fails.
Code Example:

```
const fallbackRates: Record<string, number> = {
  USD: 83,
  GBP: 105,
  EUR: 90,
  CAD: 61,
  AUD: 55
};
function getFallbackRate(
  currency: string
): number {
```

```
  const rate = fallbackRates[currency];
if (rate === undefined) {
    throw new Error(
      `Unsupported currency: ${currency}`
    );
  }
  return rate;
}
```

Solution: InvoiceBridge uses a one-hour exchange-rate cache and static fallback rates. These values are fallbacks, not current market rates. The application should indicate when a fallback is used.

4.3.3 Refactoring and Optimization

The codebase separates frontend components, backend APIs, database operations, and invoice-processing logic. TypeScript, reusable components, shared API definitions, and explicit error handling help improve maintainability.

These practices reduce duplicated logic and make future modifications easier.

4.4 Coding Prompts, Generated Code and Manual Modifications

AI-assisted coding was used to support implementation and debugging. The following examples illustrate representative coding tasks based on InvoiceBridge's documented architecture. They are not verified transcripts of the original Antigravity conversation history.

4.4.1 Database Schema

Prompt:

"Create a PostgreSQL database schema using Drizzle ORM for uploaded invoices and converted invoices. Store user ID, file name, currency, status, GST amounts and conversion details."

AI Output (Antigravity):

The suggested design separated original invoice records from conversion results using two related tables.

Code Example:

```
import {
  pgTable,
  uuid,
  text,
  timestamp,
  numeric
} from "drizzle-orm/pg-core";
```

```
export const invoices = pgTable("invoices", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: text("user_id").notNull(),
  fileName: text("file_name").notNull(),
  originalCurrency: text("original_currency"),
  status: text("status").notNull(),
  createdAt: timestamp("created_at")
    .defaultNow()
    .notNull()
});
export const convertedInvoices = pgTable(
  "converted_invoices",
  {
    id: uuid("id").defaultRandom().primaryKey(),
    invoiceId: uuid("invoice_id")
      .notNull()
      .references(() => invoices.id),
    inrAmount: numeric("inr_amount"),
    gstRate: numeric("gst_rate"),
    pdfPath: text("pdf_path"),
    processedAt: timestamp("processed_at")
      .defaultNow()
  }
);
```

Manual Modification: The actual project schema contains additional fields for extracted data, file paths, GST components, exchange rates, converted data, and processing timestamps. The code above is a simplified illustration, not the complete production schema.

4.4.2 OCR-Based Invoice Extraction

Prompt:

"Add OCR functionality so users can upload invoice images and extract text. I want users to review and correct the extracted information before conversion."

AI Output (Antigravity):

Tesseract.js can extract text from images, while PDF.js supports PDF processing. Extracted information can then be presented in editable fields.

Code Example:

```
import Tesseract from "tesseract.js";
async function extractInvoiceText(
```

```
  imagePath: string
): Promise<string> {
  const result = await Tesseract.recognize(
    imagePath,
    "eng"
  );
  return result.data.text;
}
```

Manual Modification: InvoiceBridge uses browser-based OCR with Tesseract.js and PDF.js. The extracted details are reviewed by the user because OCR can misread amounts, dates, and currency symbols.

4.4.3 GST Calculation

Prompt:

"Implement GST calculation with selectable rates of 0, 5, 12, 18 and 28 percent. Calculate CGST and SGST for intra-state supply and IGST for inter-state supply."

AI Output (Antigravity):

The proposed calculation determines the GST amount and divides it according to the supply type.

Code Example:

```
function calculateGst(
  taxableAmount: number,
  gstRate: number,
  sameState: boolean
) {
  const gst = taxableAmount * gstRate / 100;
  return {
    cgst: sameState ? gst / 2 : 0,
    sgst: sameState ? gst / 2 : 0,
    igst: sameState ? 0 : gst
  };
}
```

Manual Modification: The complete application also needs to validate invoice amounts, GST rates, and place-of-supply information before calculating tax.

5. Testing

Testing was performed as an important part of the InvoiceBridge development process to check the functionality, reliability, and correctness of the application. The testing process focused on invoice upload, OCR-based text extraction, currency conversion, GST calculation, database operations, PDF generation, and dashboard functionality.
AI assistance through Google Antigravity helped prepare test cases, analyse potential errors, and suggest debugging solutions. The application was designed to be tested at different levels to ensure that individual functions and the complete invoice conversion workflow behaved as expected.

5.1 Testing Strategy and Test Plan

The testing strategy covered individual functions, communication between modules, and the complete application workflow.

5.1.1 Testing Strategy

Testing Type	Purpose
Unit Testing	Checks individual functions such as currency conversion and GST calculation.
Integration Testing	Checks communication between the frontend, backend, database, and external services.
System Testing	Checks the complete workflow from invoice upload to PDF download.
Validation Testing	Checks invalid files, unsupported currencies, and missing input values.
Regression Testing	Checks that existing functionality continues to work after code modifications.

5.1.2 Test Plan

Test ID	Feature	Test Objective	Expected Result
TC-01	Authentication	Sign in to the application	User authentication works correctly.
TC-02	Invoice Upload	Upload a supported PDF or image	File is accepted for processing.
TC-03	File Validation	Upload an unsupported file	Invalid file is rejected.
TC-04	OCR Extraction	Extract text from an invoice	Extracted text is available for review.

TC-05	Currency Conversion	Convert foreign currency into INR	Amount is calculated using the selected rate.
TC-06	GST Calculation	Calculate intra-state GST	CGST and SGST are calculated correctly.
TC-07	GST Calculation	Calculate inter-state GST	IGST is calculated correctly.
TC-08	Database Operations	Save and retrieve invoice information	Invoice records are stored and retrieved.
TC-09	PDF Generation	Generate a converted invoice	GST invoice PDF is generated.
TC-10	Dashboard	View invoice history	Invoice records and statistics are displayed.
TC-11	PDF Download	Download the converted invoice	The generated document is downloaded.
TC-12	Email Delivery	Send an invoice through email	Email delivery is handled by the configured service.

5.2 AI-Generated Test Cases

Google Antigravity was used as an AI-assisted development tool to help prepare test scenarios for InvoiceBridge. The test cases covered normal operations, invalid inputs, calculation accuracy, and possible failures in external services.

5.2.1 Test Case Generation Prompt

Prompt:

"Generate test cases for my InvoiceBridge project. Test invoice upload, OCR extraction, currency conversion, GST calculation, database saving and PDF download. Include valid inputs, invalid inputs and expected results."

AI Output (Antigravity):

Antigravity could help organise the testing requirements into individual test cases with defined inputs and expected outcomes. These cases cover successful operations, invalid files, missing invoice details, unsupported currencies, and PDF generation errors.

5.2.2 Sample Test Cases

Test Case	Input / Condition	Expected Result
Valid invoice upload	Supported PDF file	File is accepted.
Invalid invoice upload	Unsupported file type	Validation error is displayed.

OCR extraction	Invoice containing readable text	Text is extracted for review.
Currency conversion	USD 100 at ₹83 per USD	Converted amount is ₹8,300.
Intra-state GST	Taxable value ₹1,000 at 18%	CGST ₹90 and SGST ₹90.
Inter-state GST	Taxable value ₹1,000 at 18%	IGST ₹180.
Missing invoice data	Required information is absent	Invalid data is handled appropriately.
PDF generation	Valid converted invoice data	PDF is generated.
Database failure	Database connection unavailable	Error is handled appropriately.
Unsupported currency	Currency is not supported	Conversion is rejected.

The currency and GST values are illustrative test inputs used to calculate expected results.

5.3 Unit, Integration and System Testing

Testing was divided into three main levels to evaluate individual functions, communication between modules, and the complete application.

5.3.1 Unit Testing

Unit testing checks individual functions independently. In InvoiceBridge, suitable functions include currency conversion, GST calculation, input validation, and invoice status management.

Prompt:

"Create a simple test for my GST calculation function. Check whether 18 percent GST on ₹1,000 gives the correct CGST, SGST and IGST values."

AI Output (Antigravity):

The suggested approach was to compare calculated tax values against expected results using assertions.

Code Example:

```
function calculateGst(
  amount: number,
  rate: number,
  sameState: boolean
) {
  const gst = amount * rate / 100;

  return {
    cgst: sameState ? gst / 2 : 0,
    sgst: sameState ? gst / 2 : 0,
    igst: sameState ? 0 : gst
  };
}
```

```
// Example test inputs
const intraState = calculateGst(1000, 18, true);
const interState = calculateGst(1000, 18, false);

console.log(intraState);
// { cgst: 90, sgst: 90, igst: 0 }

console.log(interState);
// { cgst: 0, sgst: 0, igst: 180 }
```

Solution: The example demonstrates the expected tax values for intra-state and inter-state supply. The complete application must also validate invoice values and the selected tax configuration.

5.3.2 Integration Testing

Integration testing checks whether the different application components work together.
The main integration checks include:

- Frontend communication with the Express API.
- Database operations through PostgreSQL and Drizzle ORM.
- Retrieval and use of exchange-rate data.
- Saving and retrieving invoice conversion results.
- Generating and downloading PDF documents.
- Authentication checks for protected invoice operations.

5.3.3 System Testing

System testing evaluates the complete InvoiceBridge workflow:

1. The user signs in.
2. The user uploads a foreign invoice.
3. The application extracts the invoice text.
4. The user reviews and corrects the extracted information.
5. The system converts the currency and calculates GST.
6. The invoice and conversion details are stored in the database.
7. The system generates the converted GST invoice.
8. The user downloads the invoice or uses the available email functionality.

The expected result is a converted invoice containing the reviewed information, INR amounts, and applicable GST components.

5.4 Defect Identification and AI-Assisted Debugging

AI-assisted debugging helped identify potential causes of errors and suggest improvements during development. The debugging process involved examining the issue, reviewing the relevant code, applying a suitable fix, and checking the affected functionality.

5.4.1 PDF Download Handling

Problem: The application needed to transfer generated PDF invoices from the backend to the frontend without binary response corruption.

Prompt:

"My generated PDF is not downloading properly from the frontend. Help me identify the problem and suggest a solution that fits my existing API."

AI Output (Antigravity):

The suggested investigation included checking response encoding, content type, proxy behaviour, and frontend file reconstruction.

Solution: InvoiceBridge uses a Base64-encoded PDF inside a JSON response from the /api/invoices/:id/download endpoint. The frontend can reconstruct the document for downloading. The API must also validate user access and handle file errors.

5.4.2 Exchange-Rate API Failure

Problem: The external exchange-rate service may be unavailable or return an error.

Prompt:

"What should I do if the exchange-rate API fails? I want to handle the error properly without crashing the whole invoice conversion process."

AI Output (Antigravity):

A suitable approach is to handle request failures, cache previously retrieved rates, and use configured fallback values when appropriate.

Solution: InvoiceBridge documents a one-hour exchange-rate cache and static fallback rates. These fallback values are not current market rates, so the application should distinguish them from live rates.

5.4.3 Debugging Process

The general debugging process involved the following activities:

1. Identifying the issue through an error message or unexpected behaviour.
2. Examining the related code and dependencies.
3. Using AI assistance to explore possible causes and solutions.
4. Modifying the code according to the project architecture.
5. Retesting the affected functionality.
6. Checking related features for regression issues.

This process supports code maintainability and helps reduce the possibility of introducing additional defects.

5.5 Testing Prompts and Actual Results

AI-assisted testing supported the preparation of test scenarios and expected outcomes for the main InvoiceBridge features. The prompts focused on authentication, invoice upload, OCR, currency conversion, GST calculation, database storage, and PDF generation.

5.5.1 Testing Prompt

Prompt:

"Review the main InvoiceBridge features and prepare test cases for authentication, invoice upload, OCR, currency conversion, GST calculation, database storage and PDF download. Include expected results for each test."

AI Output (Antigravity):

The suggested testing plan covered successful inputs, invalid inputs, missing information, external service failures, and document generation. These test cases provide a structured way to evaluate the application.

5.5.2 Test Results

Test ID	Functionality	Expected Result	Actual Result
TC-01	Authentication	User authentication works correctly.	Passed
TC-02	Invoice Upload	Supported file is accepted.	Passed
TC-03	OCR Extraction	Invoice text is extracted for review.	Passed
TC-04	Currency Conversion	INR amount matches the selected rate.	Passed
TC-05	GST Calculation	GST components match expected values.	Passed
TC-06	Database Operations	Invoice data is saved and retrieved.	Passed
TC-07	PDF Generation	Converted invoice PDF is generated.	Passed
TC-08	PDF Download	Generated PDF can be downloaded.	Passed
TC-09	Dashboard	Invoice history and statistics are displayed.	Passed
TC-10	Email Delivery	Invoice email is processed successfully.	Passed

5.5.3 Testing Summary

The testing section covers the major InvoiceBridge features, including authentication, invoice upload, OCR extraction, currency conversion, GST calculation, database operations, PDF generation, dashboard management, and email delivery. AI-assisted development supported test preparation and debugging, while the expected results provided a basis for checking application behaviour.

Important: The table above records the requested “Passed” statuses, but actual execution logs were not supplied in this conversation. Include these statuses as verified results only if the corresponding tests have been run successfully.

6. Deployment and Documentation

This chapter describes the deployment, configuration, and documentation of InvoiceBridge. The application was deployed on Vercel, with PostgreSQL hosted on Neon. Clerk, an exchange-rate API, and Resend support authentication, currency conversion, and email delivery.

6.1 Deployment and Execution Procedure

InvoiceBridge uses a monorepo structure managed through pnpm workspaces. The frontend is developed using React and Vite, while the backend uses Node.js and Express.

Deployment Technologies

Component	Technology
Frontend	React, Vite, TypeScript
Backend	Node.js, Express 5
Database	PostgreSQL on Neon
Authentication	Clerk
Hosting	Vercel
Email Service	Resend

Local Execution Steps

Step 1: Install dependencies

pnpm install

Step 2: Configure environment variables

DATABASE_URL=your_database_url

CLERK_SECRET_KEY=your_clerk_secret_key

RESEND_API_KEY=your_resend_api_key

Step 3: Start the application

pnpm run dev

The command depends on the scripts configured in the project. The documented frontend port is 22201, and the Express API port is 8080.

Step 4: Test the application

Sign in, upload an invoice, review the extracted information, perform the conversion, and download the generated GST invoice.

For production, Vercel build settings, environment variables, database connectivity, and backend deployment configuration must be correctly configured.

6.2 Deployment Prompts and Configuration

Google Antigravity assisted with deployment preparation and configuration troubleshooting. The following is a representative prompt, not a verified copy of the original prompt history.

Prompt:

"Help me deploy InvoiceBridge on Vercel. My project uses React, Vite, Node.js, Express and PostgreSQL on Neon. Explain the build settings and environment variables required."

AI Output (Antigravity):

The suggested approach was to configure the production build, identify required environment variables, verify database connectivity, and check the API routes.

Solution: The application was deployed on Vercel, with Neon providing database hosting. External service credentials were configured through environment variables rather than hardcoded in the source code.

6.3 User Manual and Technical Documentation

6.3.1 User Manual

1. Sign in: Open InvoiceBridge and authenticate using the available login options.
2. Upload invoice: Select the source country and upload a PDF, PNG, or JPG file.
3. Review extracted data: Check and correct the information extracted through OCR.
4. Convert invoice: Review the exchange rate and select the applicable GST rate.
5. Generate PDF: Generate the converted Indian GST invoice.
6. Manage invoices: Use the dashboard to view history, download invoices, and access available actions.
7. Email invoice: Use the email feature where it is configured.

6.3.2 Technical Documentation

The technical documentation covers:

- System architecture and technology stack.
- Database tables and API endpoints.
- OCR and invoice-processing workflow.
- Currency conversion and GST calculations.
- PDF generation and download.
- Deployment configuration and troubleshooting.

Summary

InvoiceBridge provides an online invoice conversion workflow supported by Vercel hosting, Neon PostgreSQL, and external services. The user manual explains the main operations, while the technical documentation supports future maintenance and development.

7. References

1. Central Board of Indirect Taxes and Customs (CBIC), Government of India, “Tax Invoice, Credit and Debit Notes.” Available: https://cbic-gst.gov.in/gst-invoice-rules.html

2. Central Board of Indirect Taxes and Customs (CBIC), Government of India, Central Goods and Services Tax Act, 2017. Available: https://cbic-gst.gov.in/

3. Central Board of Indirect Taxes and Customs (CBIC), Government of India, Integrated Goods and Services Tax Act, 2017. Available: https://cbic-gst.gov.in/

4. Organisation for Economic Co-operation and Development (OECD), International VAT/GST Guidelines. OECD Publishing, 2017. Available: https://www.oecd.org/en/publications/international-vat-gst-guidelines_9789264271401-en.html

5. European Commission, “VAT Invoicing.” Available: https://taxation-customs.ec.europa.eu/taxation/vat/vat-businesses/invoicing_en

6. HM Revenue & Customs, Government of the United Kingdom, “Invoices — What They Must Include.” Available: https://www.gov.uk/invoicing-and-taking-payment-from-customers/invoices-what-they-must-include

7. Canada Revenue Agency, Government of Canada, “Charge and Collect the GST/HST.” Available: https://www.canada.ca/en/revenue-agency/services/tax/businesses/topics/gst-hst-businesses/charge-collect-which-rate.html

8. Australian Government, business.gov.au, “How to Invoice.” Available: https://business.gov.au/finance/payments-and-invoicing/how-to-invoice

9. Reserve Bank of India (RBI), “Database on Indian Economy — Daily Interbank Forex Rates.” Available: https://dbie.rbihub.in/

8.APPENDICES

The appendices provide supporting information about the AI-assisted development, source code, interface design, testing, and deployment of InvoiceBridge.

8.1 Appendix A: Complete Prompt Dictionary and Prompt History

The following table presents representative prompts used for different development activities with Google Antigravity. The prompts and outputs are reconstructed examples, not verified copies of the original prompt history.

Prompt ID	Engineering Phase	Prompt Given to Antigravity	AI Output (Antigravity)	Manual Refinement
PR-01	Requirements	"Create a project to convert foreign invoices into Indian GST invoices."	Suggested the main modules: upload, OCR, currency conversion, GST, and PDF generation.	Organised features into separate modules.
PR-02	Database	"Create PostgreSQL tables for uploaded and converted invoices using Drizzle ORM."	Suggested related invoice and conversion tables.	Added fields for processing status, exchange rates, and GST details.
PR-03	OCR	"Extract invoice details from PDF and image files."	Suggested Tesseract.js and PDF.js for text extraction.	Added an editable review step to correct extracted information.
PR-04	Currency	"Convert foreign invoice amounts into INR and handle API errors."	Suggested exchange-rate retrieval, caching, and fallback rates.	Used the documented one-hour cache and configured fallback values.
PR-05	GST	"Calculate CGST, SGST and IGST based on the selected supply type."	Suggested separate calculations for intra-state and inter-state supplies.	Integrated the calculations with the invoice conversion workflow.
PR-06	UI Design	"Create a modern website with a home page, upload page and dashboard."	Suggested layouts and reusable interface components.	Refined the styling, spacing, and navigation.

PR-07	PDF Generation	"Generate a downloadable GST invoice PDF."	Suggested server-side PDF generation and download handling.	Integrated the PDF generation workflow using PDFKit.
PR-08	Debugging	"Help me fix PDF download and API response problems."	Suggested checking response encoding and API error handling.	Used the documented Base64 JSON response approach.
PR-09	Deployment	"Help me deploy InvoiceBridge on Vercel with PostgreSQL on Neon."	Suggested reviewing build settings and environment variables.	Configured deployment according to the project setup.

8.2 Appendix B: AI Tool Usage Log

8.2.1 AI-Assisted Development Process

Google Antigravity supported development by helping generate code, suggest improvements, and investigate errors.

Development Stage	AI Tool Contribution	Purpose
Planning	Suggested modules and workflows	Define project requirements
UI Design	Proposed page layouts	Improve usability
Coding	Generated initial code examples	Speed up implementation
Debugging	Suggested possible fixes	Investigate errors
Testing	Prepared test scenarios	Check expected behaviour
Deployment	Suggested configuration checks	Support production deployment

8.2.2 Example Code Refinements

Component	Initial Approach	Refinement
GST calculation	Basic tax calculation	Separate CGST, SGST, and IGST
Invoice records	Store basic invoice details	Include status and conversion information
OCR processing	Extract text	Allow users to review extracted fields
PDF download	Return generated document	Use Base64 data in a JSON response
Currency service	Fetch exchange rates	Add caching and fallback handling

8.3 Appendix C: Important Source Code and Screenshots

8.3.1 Important Source Code

Listing C.1: GST Calculation

```
function calculateGst(
  amount: number,
  rate: number,
  sameState: boolean
) {
  const gst = amount * rate / 100;

  return {
    cgst: sameState ? gst / 2 : 0,
    sgst: sameState ? gst / 2 : 0,
    igst: sameState ? 0 : gst
  };
}
```

Example output for ₹1,000 at 18% GST:

Intra-state:

CGST = ₹90

SGST = ₹90

IGST = ₹0

Inter-state:

CGST = ₹0

SGST = ₹0

IGST = ₹180

Listing C.2: Invoice Status Management

```
enum InvoiceStatus {
  UPLOADED = "UPLOADED",
  EXTRACTING = "EXTRACTING",
  EXTRACTED = "EXTRACTED",
  CONVERTING = "CONVERTING",
  CONVERTED = "CONVERTED",
  FAILED = "FAILED"
}
```

8.3.2 Screenshot Catalogue

Figure	Screenshot	Description
Figure C.1	Home Page	Introduces InvoiceBridge and its features.
Figure C.2	Invoice Upload	Shows country selection and file upload.
Figure C.3	Dashboard	Displays invoice history and statistics.

Figure C.4	Invoice Review	Shows extracted invoice information.
Figure C.5	Converted Invoice	Shows the generated GST invoice PDF.
Figure C.6	Deployed Website	Shows the hosted application.

8.4 Appendix D: Test Cases and Execution Evidence

8.4.1 Test Case Summary

The following is an illustrative test-results table for the main InvoiceBridge functions.

Test ID	Module	Test Scenario	Expected Result	Sample Status
TC-01	Authentication	Sign in with valid credentials	User authenticated	PASS
TC-02	Upload	Upload a PDF invoice	File accepted	PASS
TC-03	OCR	Extract invoice text	Text available for review	PASS
TC-04	Currency	Convert USD 100 at ₹83	₹8,300	PASS
TC-05	GST	Calculate 18% GST on ₹1,000	Correct tax components	PASS
TC-06	Database	Save invoice details	Record stored	PASS
TC-07	PDF	Generate GST invoice	PDF generated	PASS
TC-08	Dashboard	View invoice history	Records displayed	PASS
TC-09	Download	Download converted invoice	File downloaded	PASS
TC-10	Email	Send invoice	Email request processed	PASS

8.4.2 Sample Test Output

The following demonstrates how test results could be documented after running the project's test suite.

InvoiceBridge Test Summary

Deploying InvoiceBridge
Project: invoicebridge
Environment: Production

[1/6] Installing dependencies...
✓ Dependencies installed successfully

[2/6] Detecting framework...

✓ Framework detected: Vite

[3/6] Building application...
✓ TypeScript compilation completed
✓ Frontend assets generated

[4/6] Building server...
✓ API server build completed

[5/6] Finalizing deployment...
✓ Build output prepared successfully

[6/6] Deploying to Vercel...
✓ Deployment completed

Status: Ready
Environment: Production
Build Result: Success